Rotaract Club of Xiamen University Malaysia | Rotary Club of Kuala Lumpur Dynamics | Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: [Insert date]

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	Atlas
Project / Solution Name	Orbit, payment reconciliation for the Orbyte hub
Name (Leader)	Tharmunieshwar A/L Prakasam
Email & Phone (Leader)	tharmu34@gmail.com, 016 261 9195
University Affiliation(s)	Universiti Tunku Abdul Rahman (UTAR)
Live Prototype URL	https://orbit-casimirchiong.replit.app
Code Repository URL (public / access granted to judges)	https://github.com/Reqqa/Orbit

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

A receipt image arrives as a WhatsApp-style webhook message. In the demo this is a built-in simulator that posts Meta-shaped payloads; the real WhatsApp Cloud API path is written but has never run against Meta. The FastAPI backend stores the image and calls the extractor seam: by default a mock that looks up labelled answers by image SHA-256, optionally Claude vision, which returns the receipt fields as JSON. Deterministic Python then scores the receipt against open invoices on a 100-point scorecard, classifies it into one of eight cases and routes it to the role allowed to approve. Hub data (tenant, users and roles, customers, open invoices) is read through OrbyteConnector, currently a mock over local synthetic data. A person approves in the Next.js dashboard, which FastAPI serves as a static export on one URL, hosted on a Replit Reserved VM. The approval service records the payment, writes it and an activity event back through the connector, queues a templated customer confirmation and appends a hash-chained audit row, all in one SQLite transaction. The extractor, sender, connector and login each have one factory function, so a mock can be swapped for the real service in one line.

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
Python 3.12	Backend language, seed scripts and tests	PSF License
JavaScript / Node.js 20+ (build only)	Builds the dashboard static export; no Node process runs in production	MIT
FastAPI 0.162.0	HTTP API; also serves the built dashboard from one URL	MIT
Uvicorn 0.54.0	ASGI server the backend runs under	BSD-3-Clause
SQLAlchemy 2.0.55	ORM and transactions; every state change and its audit row commit together	MIT
SQLite (Python standard library)	Single-file database, backend/orbit.db	Public domain

Name & Version	Purpose in Project	License
Pydantic 2.14.1 / pydantic-settings 2.16.1	Request and response schemas, validating extracted receipt fields, loading .env settings	MIT
python-multipart 0.0.32	Parses the receipt image uploaded by the WhatsApp simulator	Apache-2.0
cryptography 50.0.2	Fernet encryption of the stored phone-number copy used to send replies	Apache-2.0 OR BSD-3-Clause
RapidFuzz 3.14.6	Payer-name similarity, one signal in the match scorecard	MIT
Pillow 13.0.0	Draws the 50 synthetic receipt images in the seed step	MIT-CMU
Anthropic Python SDK 2.3.0	Calls Claude vision; imported only when LLM_PROVIDER=claude	Apache-2.0
httpx 0.28.1	HTTP client for the optional live WhatsApp Cloud API path	BSD-3-Clause
pytest 9.1.1	42 tests: matching, classification, duplicate recording, RBAC, audit hash chain	MIT
Next.js 16.4.3	App Router dashboard (8 screens), built as a static export	MIT
React / React DOM 19.3.1	UI components, all hand-written (no component or chart library)	MIT
Tailwind CSS 4.3.3, @tailwindcss/postcss 4.3.3, PostCSS 8.5.31	Styling and the build-time CSS pipeline	MIT

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
Anthropic API (Claude vision), Anthropic	Reads a receipt image and returns its fields as JSON. This is the only use of AI. Optional and off by default (LLM_PROVIDER=mock), so the default demo makes no call.	Paid, pay-as-you-go with the team's own key. Not sponsor-provided.
WhatsApp Cloud API (Graph API v26.0), Meta	Sends confirmations and reminders and downloads inbound receipt images. Code is written but unverified; there is no Meta business account, so the demo uses the built-in simulator.	Not connected. Meta's free test tier is the intended route.
Orbyte hub API, Orbyte / Alpha Red	Simulated. OrbyteConnector reads tenant, users, customers and open invoices and writes back payments and activity events. Orbyte publishes no API, so the seven endpoints are our own documented assumption.	None. No call leaves the machine (mock connector).

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
Replit	Hosts the live prototype: one Reserved VM runs FastAPI, which serves the API and the static dashboard. The SQLite file is reseeded on every restart.	Reserved VM, covered by sponsor-provided Replit credits.
None other	No other cloud hosting, storage, database, authentication or compute service is used. The database is a local SQLite file.	Not applicable

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
Claude, model id claude-opus-5-5	Anthropic, via the Anthropic API	Pre-trained, used as-is. Optional; off by default (a mock lookup is used).	Anthropic API and commercial terms
None other	Not applicable	No fine-tuned or self-trained models, no embeddings, no self-hosted models (RunPod is not used).	Not applicable

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
Orbit synthetic business data: 3 tenants, 9 users, 75 customers, 80 invoices	Generated by backend/seed/generate_data.py in the repository	Synthetic	Apache-2.0 (repository)	N
Orbit synthetic receipts: 50 images and labels (ground_truth.csv)	Drawn by backend/seed/generate_receipts.py; labels in backend/seed/ground_truth.csv	Synthetic	Apache-2.0 (repository)	N
No public, purchased or real-world dataset	Not applicable	None	Not applicable	N

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☒ Yes ☐ No

If yes, list them below:

Tool & Version	**Used For**	**Extent of Use (minor / moderate / major)**
Claude Code (Anthropic), Claude Opus 5 and Claude Sonnet 5.5	Writing and reviewing code and tests, documentation, the demo script and this declaration	Major

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

Work followed docs/STEPS.md. Each step had a 'done when' check that the team ran before moving on, and Claude Code worked under the standing rules in CLAUDE.md. The team decided the specification, scoring weights, thresholds and honesty rules. 42 automated tests cover matching, classification, role-based access and the audit hash chain, and the receipt-to-paid-invoice journey was run end to end. The team reviews AI-written output and remains responsible for it.

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☒ Yes ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
Replit	Sponsor-provided credits (hosting platform): Reserved VM deployment	Deployment only. Hosts the live prototype at orbit-casimirchiong.replit.app. Not used for development.

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

Not essential. Replit is used only to deploy the finished prototype. The same code runs locally with Python 3.12 and a single SQLite file, and the application uses no Replit-specific API or database. Any host that can run one Python process could replace it, so the core functionality does not depend on Replit. RunPod and all other sponsor resources are not used.

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

The prototype uses only synthetic data: no real customers, receipts, phone numbers or bank details. Phone numbers are stored as a keyed HMAC for lookup, plus a Fernet-encrypted copy used only to send replies. Receipt images and the SQLite database stay on the host. In the default configuration nothing is sent to a third party; only if live extraction is switched on is a receipt image sent to the Anthropic API. The audit log is append-only and hash-chained. No secrets are committed; keys live in .env or Replit Secrets. The hosted demo is reseeded on every restart, so nothing persists. For a real deployment the specification (section 10) plans consent, notice, encryption, role-based access, masked account numbers and automatic deletion of receipt images after a set period, in line with Malaysia's PDPA.

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

In the default demo the receipt reader is a mock that looks up labelled answers, so its 100% extraction score is by construction and says nothing about the real model. Real extraction accuracy on live bank and wallet layouts is unmeasured. A vision model can misread an amount or invent a field, and Orbit cannot detect a forged screenshot. Mitigations: the model only reads; per-field confidence sends unclear receipts to a person; one retry then a human; nothing is recorded without role-checked approval; every action is audited. The live path depends on a single provider and is subject to rate limits and outages, but the extractor sits behind one interface and failures route to a human. Matching, scoring, approval limits and message text are deterministic and never model-written.

7. Compliance & Readiness Checklist

Tick all boxes that apply before submitting.

☒ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.

☒ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.

☒ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.

☒ Any use of generative-AI tools for development has been disclosed and reviewed by the team.

☒ Any use of official SDC sponsor resources has been declared in Section 5.

☒ The prototype described here is accessible to judges through the links provided in Section 1.

☒ All fields in this template have been completed within the word limits stated.

8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.



25 Oct 2026